Joseph Sinclair

San Mateo, CA | (650) 759-3486 | joey.sinclair02@gmail.com | github.com/joeysnclr

Education

University of California, Berkeley — Dec 2025
Bachelor of Arts in Data Science — *Berkeley, CA*

- Coursework: Data Engineering, Data Mining & Analytics, Probability, Data Structures & Algorithms, Econometrics

Technical Skills

Languages: Python, Go, SQL, TypeScript, JavaScript
Libraries & Tools: Pandas, Sci-Kit Learn, DBT, PostgreSQL, SQLite, Git, Linux
Practices: Unit Testing, ETL Pipelines, REST APIs, Feature Engineering, Statistical Modeling

Experience

Founder — 2025 – Present
Surface | surface.surf — *Remote*

- Launched cross-exchange prediction market terminal aggregating real-time contract data from Kalshi and Polymarket; shipped public match API consumed by third-party traders.
- Built hybrid contract matching system (BM25 + 768-dim vector cosine similarity) identifying equivalent markets across exchanges, surfacing arbitrage opportunities at a 0.95 confidence threshold.
- Designed idempotent 7-phase ingestion pipeline in Go handling contract fetching, FTS5 indexing, embedding generation, and LLM-powered cross-exchange mapping with concurrent workers and rate limiting.

Software Engineer Intern — June 2023 – Aug 2023
Platform Science — *San Diego, CA*

- Increased Go backend test coverage by **25%** across **29 pull requests**, focusing on Protocol Buffer serialization edge cases and concurrent goroutine/channel patterns.
- Built integration tests targeting race conditions in production business logic, requiring reliable reproduction of timing-dependent failures.

Projects

Prop Engine | *Python, dbt, XGBoost, PostgreSQL, FastAPI*

- Engineered **35+ dbt SQL models** computing rolling Statcast splits (xwOBA, hard hit rate, K rate) with handedness, pitch type, park factor, and weather adjustments across configurable lookback windows.
- Built two prediction models (XGBoost softmax, Ridge+KNN hybrid) as interchangeable pipeline components with consistent input/output schemas, outputting full probability distributions over discrete boxscore outcomes rather than point estimates.
- Designed Monte Carlo simulation framework to backtest parlay strategies across payout structures, with Kelly criterion integration for bankroll sizing.

Oracle Engine | *Go, React, SQLite, Kalshi API, Polymarket API, TradingView*

- Architected a clock abstraction layer in Go enabling identical strategy code to run in live-market and historical-backtest modes — switching is a config change, eliminating the dual-codebase maintenance problem common in trading systems.
- Integrated Dome API to source historical L2 order book snapshots for Kalshi and Polymarket contracts unavailable through official exchange APIs, enabling realistic fill simulation during backtesting.
- Built synthetic candle generation pipeline from raw tick data and a TradingView Charts dashboard reporting Sharpe ratio, max drawdown, and win rate across YAML-configured strategy runs.

Latent Jobs | *Python, FastAPI, PostgreSQL, pgvector*

- Reverse-engineered LinkedIn's private API to scrape job listings at scale; built FastAPI backend storing OpenAI embeddings (768-dim) in pgvector, ranking postings against uploaded resumes by cosine similarity rather than keyword overlap.
- Built normalization pipeline converting unstructured job descriptions into typed PostgreSQL records (tech stack, seniority, salary, YOE), enabling structured filtering on top of vector similarity ranking.